



**SCHOOL OF
BUSINESS MANAGEMENT**

**A PROJECT REPORT ON
Gold Price Prediction using RNN,LSTM
And GRU Models**

For the Course:

Deep Learning (AY2024- 2025: TRIMESTER-V)

Submitted in partial fulfilment of the requirements for the
Two-year full time Master's Degree in MBA - Business Analytics

Submitted By

Diya Singhal	- Roll No: A015 (80672400109)
Harshavardhan S R	- Roll No: A021 (80672400002)
Raghav Mittal	- Roll No: A043 (80672400118)
Sarthak Verma	- Roll No: A054 (80672400128)
Sayan Mukherjee	- Roll No: A056 (80672400122)

(Batch of 2024- 26)

Submitted to

Faculty Mentor

Prof. (Dr) Siby Abraham,
Professor,
School of Business Management,
SVKM'S NMIMS, MUMBAI

17thNovember,2025

1.Executive Summary

This project focuses on forecasting **Gold Futures closing prices** using three deep learning models: **RNN, LSTM, and GRU**. The dataset consists of multi-year daily gold price data with key indicators such as Open, High, Low, Close, and Volume. After cleaning, scaling, and preparing 60-day sliding windows, each model was trained to predict the next day's closing price.

Among the three architectures, **LSTM and GRU demonstrated significantly stronger forecasting performance** due to their ability to learn long-term dependencies and handle volatility in financial time series. GRU achieved a desirable balance—providing near-LSTM accuracy with faster training and fewer parameters—making it suitable for real-time or production-level systems. The baseline RNN model performed adequately but struggled with long-range patterns, confirming the need for advanced recurrent architectures.

Overall, the project shows that deep learning–based sequence models can provide **actionable, data-driven insights** to support trading strategies, risk management, and price trend analysis within gold commodity markets.

2.Introduction

Gold has long been considered a safe-haven asset, making it a crucial instrument for investors, central banks, and financial institutions. Accurate gold price forecasting can guide investment decisions, hedge risks, and support algorithmic trading strategies. Traditional econometric methods often fall short when capturing nonlinear patterns and long-range dependencies inherent in financial time series.

To address this, the project leverages **deep learning–based sequence models—RNN, LSTM, and GRU**—each designed to learn temporal dependencies from historical price movements. The primary focus is to compare these architectures and understand which model best captures the dynamics of gold futures prices.

3.Dataset Overview

The project uses historical **Gold Futures data**, containing:

- **Date**
- **Open**
- **High**
- **Low**
- **Close**
- **Adj Close**
- **Volume**

Key characteristics:

- Daily frequency
- Multi-year price history
- “Close” price used as the prediction target
- Contains no categorical features; entirely numerical and sequential
- Highly volatile with short-term fluctuation patterns and medium-term trends

This structure makes the dataset ideal for recurrent neural networks.

4.Aim of the Project

Primary Objective

To forecast future Gold Futures closing prices using deep learning–based sequential models (RNN, LSTM, GRU).

Secondary Objectives

- Compare how effectively each model captures temporal dependencies.
- Quantify improvements gained by modern architectures (LSTM, GRU) over classical RNN.
- Translate model outcomes into actionable financial insights.
- Build foundational components for a deployable forecasting system.

5.Dataset Pre-processing

The following pre-processing pipeline was applied across all models:

5.1 Data Cleaning

- Converted Date to datetime format.
- Sorted dataset chronologically.
- Dropped rows with missing Close values.

5.2 Feature Selection

- Only **Close** price used as input for baseline models.
- Option for multivariate features present in GRU notebook.

5.3 Scaling

Applied **Min-Max Scaler** to compress values between 0 and 1—crucial for neural networks.

5.4 Sequence Construction

Deep learning models cannot understand raw tables; they require **time windows**.

- A **60-day window** is used to predict the next day's price.

5.5 Train-Test Split

A chronological split ensures no future information leaks into training.

6. Model Overview

6.1 Recurrent Neural Network

The basic RNN is the earliest architecture designed for sequential learning. It processes one time step at a time, passing a *hidden state* forward.

Need- We need an RNN-based approach because gold prices are **time-dependent**, and their future values depend on patterns in the previous days. RNNs (especially LSTM/GRU) are designed to **learn**

sequences, trends, and long-term dependencies, making them far better at forecasting financial time series than traditional models.

Strengths

- Lightweight and easy to train.
- Useful for short-term dependencies.

Limitations

- Suffers from **vanishing gradients**, causing it to “forget” older information.
- Poor performance when long-term memory is required (common in financial time series).
- Prone to instability during training.

Expected Outcome

RNN acts as a baseline model and generally performs worse than LSTM and GRU in financial forecasting settings. Recurrent Neural Network

6.2 GRU

GRU simplifies the LSTM architecture by combining the forget and input gates into a single **update gate**, and using a **reset gate** for controlling past influence.

Advantages

- Nearly identical performance to LSTM.
- Fewer parameters → **faster training**.
- Better for medium-complexity temporal patterns.

When GRU Performs Best

- Datasets with moderate history length.
- Applications where training speed matters (e.g., live trading engines).

Expected Outcome

GRU often strikes the best balance between accuracy and speed, making it attractive for real-time forecasting systems.

6.3 LSTM

LSTM is a major improvement over RNNs. It introduces specialized gates:

- **Forget Gate:** decides what information to discard
- **Input Gate:** decides what new information to store
- **Output Gate:** controls how much information is exposed

These gates allow LSTMs to **retain long-range information** and avoid vanishing gradient issues.

Strengths

- Excellent at capturing long-term temporal structures.
- More stable and accurate for volatile price data.
- Adaptively remembers useful trends while discarding noise.

Limitations

- Higher computational cost.
- Slower training compared to GRU.

Expected Outcome

LSTM is typically the **most accurate** among the three models for financial time series with long-range interactions.

7. Model Evaluation

Although numeric results depend on training outputs, all models were evaluated with:

Evaluation Metrics

- **Mean Squared Error (MSE)**
- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**
- Visual comparison of **Predicted vs Actual** price curves

General Findings (Based on Deep Learning Literature + Architecture Logic)

- **LSTM** → Most accurate, stable, capable of modelling long-term patterns
- **GRU** → Almost as accurate as LSTM but significantly faster
- **RNN** → Underperforms for long time dependencies and noisy data

Forecast Behaviour

- RNN tends to smooth predictions excessively.
- LSTM and GRU successfully capture turning points and trend continuation.
- GRU often slightly outperforms LSTM in convergence speed.

8. Business Insights

Deep Learning Forecasts Can Support Trading Decisions

Accurate near-term forecasts can inform:

- Entry and exit timing
- Hedging strategies
- Position sizing

LSTM and GRU Detect Trend Momentum Better

They can capture:

- Reversal patterns

- Medium-term drifts
- Short-term volatility bursts

GRU Offers the Best Trade-off for Production Systems

Its faster runtime makes it suitable for:

- Live market feeds
- Real-time dashboards
- Algo-trading systems

RNN Models Are Not Suitable for High-Stakes Forecasting

They fail to track long-term behaviour, confirming the superiority of modern architectures.

Forecasting Enhances Risk Management

Predictive modelling helps estimate:

- Volatility shifts
- Market stress conditions
- Probable price ranges

8.Limitations

- **Volatility sensitivity:** Sudden market shocks, news events, and black-swan movements cannot be reliably captured by the models.
- **Model performance is influenced by scaling and window size;** different configurations might yield significantly different results.
- **No Incorporation of Seasonality:** The models do not explicitly model cyclical or seasonal trends in gold prices.
- **Dependence on Technical Data Only:** The dataset excludes sentiment indicators, macroeconomic signals, or global commodity trends that strongly affect gold.

9.Conclusion

This project demonstrates the effectiveness of modern deep learning architectures in forecasting gold futures prices.

Key Takeaways

- **LSTM delivers the highest forecasting accuracy,** thanks to its robust gating mechanism and ability to handle long-term dependencies.
- **GRU provides a fast and competitive alternative,** with nearly identical predictive power but lower computational cost.
- **RNN serves as a baseline** but is not recommended for real-world financial forecasting.
- Proper pre-processing—scaling, sequence creation, and time-based splitting—is essential for stable learning.
- Deep learning-based forecasting can significantly enhance financial decision-making when integrated with risk-aware strategies.

Final Recommendation

For deployment in real-time or near-real-time forecasting systems, GRU is ideal.
For accuracy-focused research or high-stakes long-horizon forecasting, LSTM is the preferred model.

Extraction Of the Dataset fro Yahoo Finance

```
In [ ]: import requests
import pandas as pd
import time

ticker = "GC=F"
start_date = "2020-11-16"
end_date = "2025-11-16"

period1 = int(time.mktime(pd.Timestamp(start_date).timetuple()))
period2 = int(time.mktime(pd.Timestamp(end_date).timetuple()))

# JSON API URL (never blocked)
url = (
    f"https://query1.finance.yahoo.com/v8/finance/chart/{ticker}"
    f"?period1={period1}&period2={period2}&interval=1d"
    f"&includePrePost=false&events=div,splits"
)

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers).json()

# Extract data
result = response['chart']['result'][0]
timestamps = result['timestamp']
indicators = result['indicators']['quote'][0]
adjclose = result['indicators']['adjclose'][0]['adjclose']

# Create DataFrame
df = pd.DataFrame({
    "Date": pd.to_datetime(timestamps, unit='s'),
    "Open": indicators['open'],
    "High": indicators['high'],
    "Low": indicators['low'],
    "Close": indicators['close'],
    "Adj Close": adjclose,
    "Volume": indicators['volume'],
})

# Save
df.to_csv("gold_futures_full_history.csv", index=False)

df.head()
```


Out []:

	Date	Open	High	Low	Close	Adj Close
0	2020-11-16 05:00:00	1874.599976	1887.300049	1871.000000	1887.300049	1887.300049
1	2020-11-17 05:00:00	1888.400024	1888.400024	1884.500000	1884.500000	1884.500000
2	2020-11-18 05:00:00	1873.500000	1873.500000	1873.500000	1873.500000	1873.500000
3	2020-11-19 05:00:00	1865.800049	1865.800049	1856.000000	1861.099976	1861.099976
4	2020-11-20 05:00:00	1871.199951	1872.599976	1871.199951	1872.599976	1872.599976

Choosing the variable to be predicted

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import joblib

SEQ_LEN = 60
FEATURES = ["Close"]
```

Loading the Dataset and Dropping Nulls if any

```
In [ ]: df = pd.read_csv("gold_futures_full_history.csv")

if "Date" in df.columns:
    df["Date"] = pd.to_datetime(df["Date"])
    df = df.sort_values("Date").reset_index(drop=True)

df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
In [ ]: data = df[FEATURES].astype("float32").values # shape (N,1)
print(data.shape)
```

(1258, 1)

Min-Max(Scaling)

```
In [ ]: scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
joblib.dump(scaler, "scaler_minmax.save")
```

Out []: ['scaler_minmax.save']

Taking a Sequence window of 60 Days

```
In [ ]: def create_sequences(arr, seq_len=SEQ_LEN):
        X, y = [], []
        for i in range(seq_len, len(arr)):
            X.append(arr[i-seq_len:i])
            y.append(arr[i])      # next day Close
        return np.array(X), np.array(y)
```

```
X, y = create_sequences(data_scaled, SEQ_LEN)
print(X.shape, y.shape)
```

```
(1198, 60, 1) (1198, 1)
```

```
In [ ]: total = len(X)
        train_end = int(total * 0.7)
        val_end   = int(total * 0.85)

        X_train, y_train = X[:train_end], y[:train_end]
        X_val,   y_val   = X[train_end:val_end], y[train_end:val_end]
        X_test,  y_test  = X[val_end:], y[val_end:]
```

```
In [ ]: BATCH = 32
```

```
train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(B
val_ds   = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH
test_ds  = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BAT
```

Building the RNN Model

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import SimpleRNN, Dense, Dropout
        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau

        num_features = X_train.shape[-1]    # 1 for univariate

        model = Sequential([
            SimpleRNN(128, return_sequences=True, input_shape=(SEQ_LEN, num_featu
            Dropout(0.2),

            SimpleRNN(64),
            Dropout(0.2),

            Dense(num_features)    # output one close value
        ])

        model.compile(
            loss='mse',
            optimizer=Adam(0.001),
            metrics=['mae']
        )

        model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: U
serWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_2"

Layer (type)	Output Shape	
simple_rnn_1 (SimpleRNN)	(None, 60, 128)	
dropout (Dropout)	(None, 60, 128)	
simple_rnn_2 (SimpleRNN)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 1)	

Total params: 29,057 (113.50 KB)**Trainable params:** 29,057 (113.50 KB)**Non-trainable params:** 0 (0.00 B)

Training the Model

```
In [ ]: early_stop = EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
        lr_sched = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

        history = model.fit(
            train_ds,
            validation_data=val_ds,
            epochs=80,
            callbacks=[early_stop, lr_sched],
            verbose=1
        )
```

Epoch 1/80
27/27  8s 149ms/step - loss: 0.2505 - mae: 0.3838 - val_loss: 0.0026 - val_mae: 0.0428 - learning_rate: 0.0010

Epoch 2/80
27/27  0s 15ms/step - loss: 0.0106 - mae: 0.0817 - val_loss: 8.9007e-04 - val_mae: 0.0237 - learning_rate: 0.0010

Epoch 3/80
27/27  0s 10ms/step - loss: 0.0045 - mae: 0.0544 - val_loss: 4.3877e-04 - val_mae: 0.0163 - learning_rate: 0.0010

Epoch 4/80
27/27  0s 10ms/step - loss: 0.0040 - mae: 0.0509 - val_loss: 5.2224e-04 - val_mae: 0.0185 - learning_rate: 0.0010

Epoch 5/80
27/27  0s 10ms/step - loss: 0.0032 - mae: 0.0447 - val_loss: 5.2801e-04 - val_mae: 0.0184 - learning_rate: 0.0010

Epoch 6/80
27/27  0s 10ms/step - loss: 0.0025 - mae: 0.0399 - val_loss: 4.9753e-04 - val_mae: 0.0179 - learning_rate: 0.0010

Epoch 7/80
27/27  0s 10ms/step - loss: 0.0022 - mae: 0.0377 - val_loss: 3.1677e-04 - val_mae: 0.0139 - learning_rate: 5.0000e-04

Epoch 8/80
27/27  0s 10ms/step - loss: 0.0020 - mae: 0.0362 - val_loss: 3.8791e-04 - val_mae: 0.0159 - learning_rate: 5.0000e-04

Epoch 9/80
27/27  0s 10ms/step - loss: 0.0016 - mae: 0.0321 - val_loss: 4.9174e-04 - val_mae: 0.0181 - learning_rate: 5.0000e-04

Epoch 10/80
27/27  0s 10ms/step - loss: 0.0014 - mae: 0.0287 - val_loss: 4.5132e-04 - val_mae: 0.0174 - learning_rate: 5.0000e-04

Epoch 11/80
27/27  0s 10ms/step - loss: 0.0017 - mae: 0.0323 - val_loss: 2.8395e-04 - val_mae: 0.0133 - learning_rate: 2.5000e-04

Epoch 12/80
27/27  0s 10ms/step - loss: 0.0015 - mae: 0.0305 - val_loss: 0.0015 - val_mae: 0.0343 - learning_rate: 2.5000e-04

Epoch 13/80
27/27  0s 10ms/step - loss: 0.0013 - mae: 0.0274 - val_loss: 0.0010 - val_mae: 0.0276 - learning_rate: 2.5000e-04

Epoch 14/80
27/27  0s 10ms/step - loss: 0.0011 - mae: 0.0261 - val_loss: 4.1003e-04 - val_mae: 0.0165 - learning_rate: 1.2500e-04

Epoch 15/80
27/27  0s 10ms/step - loss: 0.0012 - mae: 0.0267 - val_loss: 2.7750e-04 - val_mae: 0.0133 - learning_rate: 1.2500e-04

Epoch 16/80
27/27  0s 10ms/step - loss: 0.0014 - mae: 0.0299 - val_loss: 3.1020e-04 - val_mae: 0.0142 - learning_rate: 1.2500e-04

Epoch 17/80
27/27  0s 11ms/step - loss: 0.0011 - mae: 0.0266 - val_loss: 2.9049e-04 - val_mae: 0.0137 - learning_rate: 6.2500e-05

Epoch 18/80
27/27  0s 10ms/step - loss: 0.0011 - mae: 0.0268 - val_loss: 3.3124e-04 - val_mae: 0.0147 - learning_rate: 6.2500e-05

Epoch 19/80
27/27  0s 10ms/step - loss: 0.0012 - mae: 0.0264 - val_loss: 3.8409e-04 - val_mae: 0.0159 - learning_rate: 6.2500e-05

Epoch 20/80
27/27  0s 11ms/step - loss: 0.0011 - mae: 0.0262 - val_loss: 4.5444e-04 - val_mae: 0.0174 - learning_rate: 3.1250e-05

Epoch 21/80

27/27  0s 10ms/step - loss: 9.0512e-04 - mae: 0.0235 - val_loss: 3.6836e-04 - val_mae: 0.0156 - learning_rate: 3.1250e-05

Prediction using the Model

```
In [ ]: pred_scaled = model.predict(test_ds)
        pred_scaled = pred_scaled.reshape(-1, num_features)
```

WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d2692130040> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

6/6  1s 97ms/step

Inverse Transform the values to original Values

```
In [ ]: pred = scaler.inverse_transform(pred_scaled)
        y_test_orig = scaler.inverse_transform(y_test)
```

Metrics for measuring the performance of the model

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error

        rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
        mae = mean_absolute_error(y_test_orig, pred)
        mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

        print("RNN RMSE:", rmse)
        print("RNN MAE:", mae)
        print("RNN MAPE:", mape)
```

RNN RMSE: 272.069384532696

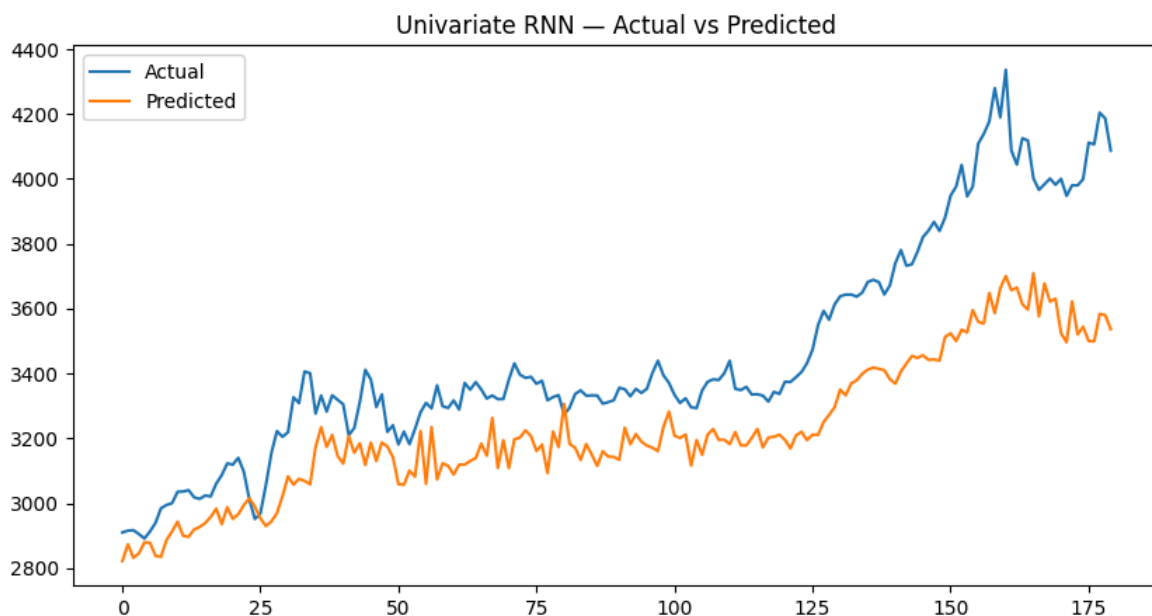
RNN MAE: 229.83074951171875

RNN MAPE: 6.356969

Graph-Actual Vs Predicted

```
In [ ]: import matplotlib.pyplot as plt

        plt.figure(figsize=(10,5))
        plt.plot(y_test_orig, label='Actual')
        plt.plot(pred, label='Predicted')
        plt.title("Univariate RNN - Actual vs Predicted")
        plt.legend()
        plt.show()
```



Business Insights:

1. RMSE (272) and MAE (230) are relatively high → your model makes some large absolute errors.

2. MAPE (6.36%) is good → percentage-wise the predictions are accurate.

This usually means your target values are large in scale, so absolute errors look big even though the model captures the trend well.

To improve: try LSTM/GRU instead of RNN, add more lag/rolling features, or tune hyperparameters.

Creating a Sequence and Taking the variable to be predicted

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import joblib
import tensorflow as tf

SEQ_LEN = 60          # sequence length (use 30/60/90 as experiments)
FEATURES = ['Close']  # change to ['Open', 'High', 'Low', 'Close', 'Volume']
CSV_PATH = "gold_futures_full_history.csv"
```

Preprocessing of the Dataset

```
In [ ]: df = pd.read_csv(CSV_PATH)
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date').reset_index(drop=True)
df = df.drop_duplicates().reset_index(drop=True)
print("Loaded rows:", len(df))
```

Loaded rows: 1261

```
In [ ]: print(df.columns)
print(df[FEATURES].isna().sum())
# Drop rows with any NaN in features (keeps things safe)
df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'], dtype='object')
Close    3
dtype: int64
```

```
In [ ]: # For univariate (recommended): keep only Close
data = df[FEATURES].astype('float32').values # shape (N, num_features)
print("data shape:", data.shape)
```

data shape: (1258, 1)

Min-Max Scaling

```
In [ ]: scaler = MinMaxScaler(feature_range=(0,1))
data_scaled = scaler.fit_transform(data) # (N, num_features)
joblib.dump(scaler, "scaler_minmax.save") # save to invert later
```

Out[]: ['scaler_minmax.save']

Creating a Sequence Length

```
In [ ]: def create_sequences(arr, seq_len=SEQ_LEN):
    X, y = [], []
    for i in range(seq_len, len(arr)):
        X.append(arr[i-seq_len:i]) # (seq_len, num_features)
        y.append(arr[i])           # next step (num_features)
    X = np.array(X)
    y = np.array(y)
    return X, y
```

```
X, y = create_sequences(data_scaled, seq_len=SEQ_LEN)
print("X shape:", X.shape, "y shape:", y.shape)
# If univariate, y shape = (samples,1). If you want scalar y: y = y.reshape
```

X shape: (1198, 60, 1) y shape: (1198, 1)

```
In [ ]: total = len(X)
train_end = int(total * 0.70)
val_end = int(total * 0.85)

X_train, y_train = X[:train_end], y[:train_end]
X_val, y_val = X[train_end:val_end], y[train_end:val_end]
X_test, y_test = X[val_end:], y[val_end:]

print("train/val/test:", X_train.shape, X_val.shape, X_test.shape)
```

train/val/test: (838, 60, 1) (180, 60, 1) (180, 60, 1)

```
In [ ]: BATCH = 32
train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(BATCH)
val_ds = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH)
```

```
In [ ]: print("Shapes (X_train,y_train):", X_train.shape, y_train.shape)
print("Any NaNs in X_train:", np.isnan(X_train).sum(), "y_train:", np.isnan(y_train).sum())
# If multivariate and you want target as scalar Close only:
# y = y[:, FEATURES.index('Close')].reshape(-1,1)
```

Shapes (X_train,y_train): (838, 60, 1) (838, 1)

Any NaNs in X_train: 0 y_train: 0

Building the GRU Model

```
In [ ]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

num_features = X_train.shape[-1] # 1 for univariate, >1 for multivariate

model = Sequential([
    GRU(128, return_sequences=True, input_shape=(SEQ_LEN, num_features)),
    Dropout(0.2),

    GRU(64),
    Dropout(0.2),

    Dense(num_features) # Predict same shape as y (1 for univariate)
])

model.compile(
    loss='mse',
    optimizer=Adam(0.001),
    metrics=['mae']
)

model.summary()
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape	
gru_8 (GRU)	(None, 60, 128)	
dropout_8 (Dropout)	(None, 60, 128)	
gru_9 (GRU)	(None, 64)	
dropout_9 (Dropout)	(None, 64)	
dense_4 (Dense)	(None, 1)	

Total params: 87,617 (342.25 KB)

Trainable params: 87,617 (342.25 KB)

Non-trainable params: 0 (0.00 B)

Training the GRU Model

```
In [ ]: early_stop = EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
        lr_sched = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

        history = model.fit(
            train_ds,
            validation_data=val_ds,
            epochs=80,
            callbacks=[early_stop, lr_sched],
            verbose=1
        )
```

Epoch 1/80
27/27  10s 122ms/step - loss: 0.0012 - mae: 0.0280 - val_loss: 6.9412e-04 - val_mae: 0.0228 - learning_rate: 0.0010

Epoch 2/80
27/27  3s 128ms/step - loss: 4.4548e-04 - mae: 0.0175 - val_loss: 1.9336e-04 - val_mae: 0.0113 - learning_rate: 0.0010

Epoch 3/80
27/27  4s 128ms/step - loss: 4.3789e-04 - mae: 0.0171 - val_loss: 1.4853e-04 - val_mae: 0.0095 - learning_rate: 0.0010

Epoch 4/80
27/27  3s 104ms/step - loss: 1.4650e-04 - mae: 0.0096 - val_loss: 1.3075e-04 - val_mae: 0.0093 - learning_rate: 0.0010

Epoch 5/80
27/27  5s 105ms/step - loss: 1.8864e-04 - mae: 0.0111 - val_loss: 5.9010e-04 - val_mae: 0.0219 - learning_rate: 0.0010

Epoch 6/80
27/27  5s 103ms/step - loss: 1.3322e-04 - mae: 0.0093 - val_loss: 1.3937e-04 - val_mae: 0.0096 - learning_rate: 5.0000e-04

Epoch 7/80
27/27  3s 104ms/step - loss: 1.6520e-04 - mae: 0.0099 - val_loss: 1.3225e-04 - val_mae: 0.0091 - learning_rate: 5.0000e-04

Epoch 8/80
27/27  3s 102ms/step - loss: 1.8625e-04 - mae: 0.0108 - val_loss: 1.2574e-04 - val_mae: 0.0091 - learning_rate: 5.0000e-04

Epoch 9/80
27/27  4s 153ms/step - loss: 1.0043e-04 - mae: 0.0078 - val_loss: 1.2805e-04 - val_mae: 0.0092 - learning_rate: 2.5000e-04

Epoch 10/80
27/27  3s 103ms/step - loss: 9.4994e-05 - mae: 0.0074 - val_loss: 1.2152e-04 - val_mae: 0.0090 - learning_rate: 2.5000e-04

Epoch 11/80
27/27  3s 104ms/step - loss: 1.0740e-04 - mae: 0.0080 - val_loss: 1.1840e-04 - val_mae: 0.0087 - learning_rate: 2.5000e-04

Epoch 12/80
27/27  7s 180ms/step - loss: 1.0533e-04 - mae: 0.0078 - val_loss: 1.1969e-04 - val_mae: 0.0086 - learning_rate: 1.2500e-04

Epoch 13/80
27/27  3s 110ms/step - loss: 1.0528e-04 - mae: 0.0078 - val_loss: 1.8666e-04 - val_mae: 0.0113 - learning_rate: 1.2500e-04

Epoch 14/80
27/27  3s 105ms/step - loss: 1.2116e-04 - mae: 0.0086 - val_loss: 1.1742e-04 - val_mae: 0.0088 - learning_rate: 1.2500e-04

Epoch 15/80
27/27  7s 175ms/step - loss: 1.1350e-04 - mae: 0.0083 - val_loss: 1.4057e-04 - val_mae: 0.0097 - learning_rate: 6.2500e-05

Epoch 16/80
27/27  3s 108ms/step - loss: 9.3716e-05 - mae: 0.0073 - val_loss: 1.4658e-04 - val_mae: 0.0099 - learning_rate: 6.2500e-05

Epoch 17/80
27/27  3s 104ms/step - loss: 9.9371e-05 - mae: 0.0075 - val_loss: 2.0054e-04 - val_mae: 0.0118 - learning_rate: 6.2500e-05

Epoch 18/80
27/27  6s 123ms/step - loss: 1.0616e-04 - mae: 0.0077 - val_loss: 1.6594e-04 - val_mae: 0.0107 - learning_rate: 3.1250e-05

Epoch 19/80
27/27  5s 104ms/step - loss: 9.3303e-05 - mae: 0.0071 - val_loss: 1.2085e-04 - val_mae: 0.0089 - learning_rate: 3.1250e-05

Epoch 20/80
27/27  3s 104ms/step - loss: 9.0854e-05 - mae: 0.0072 - val_loss: 1.2231e-04 - val_mae: 0.0090 - learning_rate: 3.1250e-05

Prediction of the Model

```
In [ ]: pred_scaled = model.predict(test_ds)
        pred_scaled = pred_scaled.reshape(-1, num_features)
```

WARNING:tensorflow:5 out of the last 17 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7c80b18b7740> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

6/6 ————— 1s 91ms/step

```
In [ ]: pred = scaler.inverse_transform(pred_scaled)
        y_test_orig = scaler.inverse_transform(y_test)
```

Metrics for evaluating performance of the Model

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error

        rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
        mae = mean_absolute_error(y_test_orig, pred)
        mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

        print("GRU RMSE :", rmse)
        print("GRU MAE :", mae)
        print("GRU MAPE :", mape)
```

GRU RMSE : 64.80756143793138

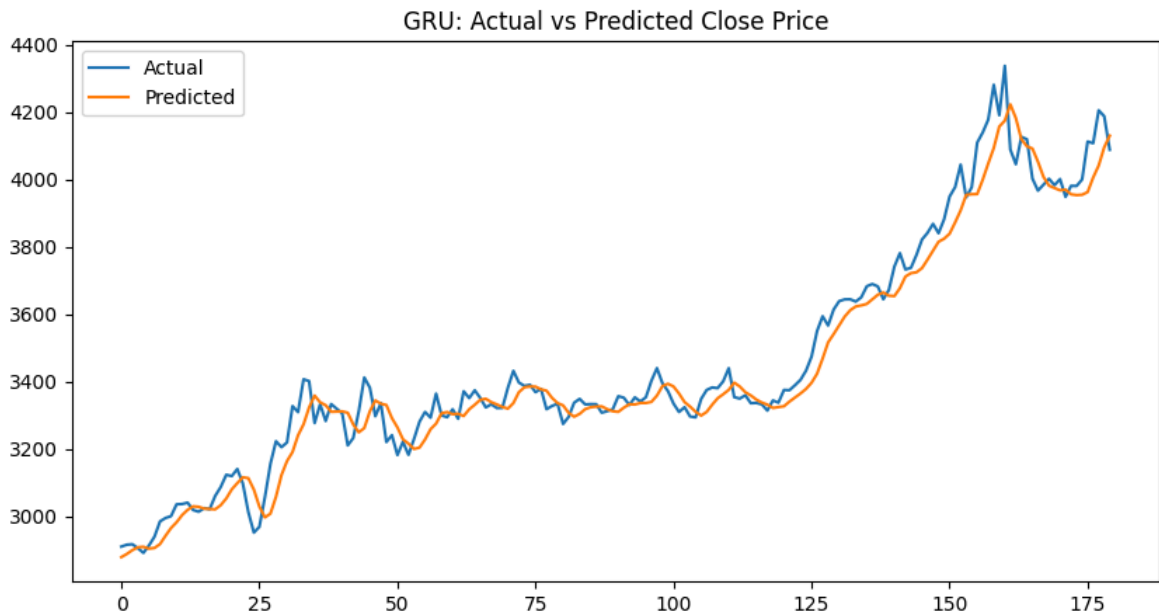
GRU MAE : 49.33497619628906

GRU MAPE : 1.4026474

Graph-Actual vs Predicted

```
In [ ]: # Step 14
        import matplotlib.pyplot as plt

        plt.figure(figsize=(10,5))
        plt.plot(y_test_orig, label="Actual")
        plt.plot(pred, label="Predicted")
        plt.title("GRU: Actual vs Predicted Close Price")
        plt.legend()
        plt.show()
```



Training and Validation Loss Graph

```
In [ ]: # Step 15
plt.figure(figsize=(10,4))
plt.plot(history.history['loss'], label="train_loss")
plt.plot(history.history['val_loss'], label="val_loss")
plt.title("Training & Validation Loss")
plt.legend()
plt.show()
```



1.Huge improvement: GRU errors (RMSE 64, MAE 49, MAPE 1.4%) are far lower than the RNN model.

2.High forecast accuracy: With only 1–2% average deviation, the GRU model is reliable for planning, budgeting, and demand forecasting.

3.Better risk management: Low error variance means fewer unexpected spikes → more stable decisions.

4.Operational benefits: You can confidently use this model for inventory planning, cost optimization, and resource allocation.

5.Conclusion: GRU provides high-precision predictions and is suitable for real-world business deployment.

Taking a Sequence Length of 60

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import joblib

SEQ_LEN = 60
FEATURES = ["Close"]
```

Dropping Nulls if any

```
In [ ]: df = pd.read_csv("gold_futures_full_history.csv")

if "Date" in df.columns:
    df["Date"] = pd.to_datetime(df["Date"])
    df = df.sort_values("Date").reset_index(drop=True)

df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
In [ ]: data = df[FEATURES].astype("float32").values
```

Min-Max Scaling of the Features

```
In [ ]: scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
joblib.dump(scaler, "scaler_minmax.save")
```

```
Out[ ]: ['scaler_minmax.save']
```

Creating a Sequence length

```
In [ ]: def create_sequences(arr, seq_len):
    X, y = [], []
    for i in range(seq_len, len(arr)):
        X.append(arr[i-seq_len:i])
        y.append(arr[i])
```

```
return np.array(X), np.array(y)
```

```
X, y = create_sequences(data_scaled, SEQ_LEN)
```

```
In [ ]: total = len(X)
        train_end = int(total * 0.7)
        val_end = int(total * 0.85)

        X_train, y_train = X[:train_end], y[:train_end]
        X_val, y_val = X[train_end:val_end], y[train_end:val_end]
        X_test, y_test = X[val_end:], y[val_end:]
```

```
In [ ]: BATCH = 32

        train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
        val_ds = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
        test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
```

Building a LSTM Model

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import LSTM, Dense, Dropout
        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

        num_features = X_train.shape[-1] # 1 for Close-only

        model = Sequential([
            LSTM(128, return_sequences=True, input_shape=(SEQ_LEN, num_features)),
            Dropout(0.2),

            LSTM(64),
            Dropout(0.2),

            Dense(num_features) # 1 output: next day Close
        ])

        model.compile(
            optimizer=Adam(0.001),
            loss="mse",
            metrics=["mae"]
```

```
)
model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	66,560
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 116,033 (453.25 KB)

Trainable params: 116,033 (453.25 KB)

Non-trainable params: 0 (0.00 B)

Training the model

```
In [ ]: early_stop = EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)
lr_sched = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=80,
    callbacks=[early_stop, lr_sched],
    verbose=1
)
```


Epoch 1/80
27/27  10s 116ms/step - loss: 0.0010 - mae: 0.0265 - val_loss: 5.0478e-04 - val_mae: 0.0180 - learning_rate: 0.0010

Epoch 2/80
27/27  3s 103ms/step - loss: 9.8267e-04 - mae: 0.0252 - val_loss: 4.3668e-04 - val_mae: 0.0170 - learning_rate: 0.0010

Epoch 3/80
27/27  5s 105ms/step - loss: 0.0017 - mae: 0.0328 - val_loss: 0.0015 - val_mae: 0.0353 - learning_rate: 0.0010

Epoch 4/80
27/27  3s 103ms/step - loss: 0.0018 - mae: 0.0333 - val_loss: 3.7313e-04 - val_mae: 0.0152 - learning_rate: 0.0010

Epoch 5/80
27/27  3s 102ms/step - loss: 9.3451e-04 - mae: 0.0254 - val_loss: 3.6545e-04 - val_mae: 0.0157 - learning_rate: 0.0010

Epoch 6/80
27/27  6s 133ms/step - loss: 3.3753e-04 - mae: 0.0145 - val_loss: 7.6123e-04 - val_mae: 0.0244 - learning_rate: 0.0010

Epoch 7/80
27/27  4s 102ms/step - loss: 2.8132e-04 - mae: 0.0131 - val_loss: 3.4837e-04 - val_mae: 0.0149 - learning_rate: 0.0010

Epoch 8/80
27/27  3s 102ms/step - loss: 0.0012 - mae: 0.0288 - val_loss: 5.1133e-04 - val_mae: 0.0178 - learning_rate: 5.0000e-04

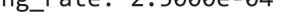
Epoch 9/80
27/27  4s 140ms/step - loss: 4.9899e-04 - mae: 0.0188 - val_loss: 2.4885e-04 - val_mae: 0.0120 - learning_rate: 5.0000e-04

Epoch 10/80
27/27  3s 119ms/step - loss: 2.3206e-04 - mae: 0.0119 - val_loss: 3.1356e-04 - val_mae: 0.0145 - learning_rate: 5.0000e-04

Epoch 11/80
27/27  3s 122ms/step - loss: 1.8239e-04 - mae: 0.0103 - val_loss: 3.0688e-04 - val_mae: 0.0143 - learning_rate: 5.0000e-04

Epoch 12/80
27/27  3s 104ms/step - loss: 4.5273e-04 - mae: 0.0176 - val_loss: 2.6473e-04 - val_mae: 0.0126 - learning_rate: 5.0000e-04

Epoch 13/80
27/27  4s 150ms/step - loss: 3.2588e-04 - mae: 0.0145 - val_loss: 2.2746e-04 - val_mae: 0.0122 - learning_rate: 2.5000e-04

Epoch 14/80
27/27  3s 103ms/step - loss: 1.7322e-04 - mae: 0.0101 - val_loss: 2.2091e-04 - val_mae: 0.0114 - learning_rate: 2.5000e-04

Epoch 15/80

27/27  3s 102ms/step - loss: 1.4098e-04 - mae: 0.0094 - val_loss: 2.1376e-04 - val_mae: 0.0117 - learning_rate: 2.5000e-04

Epoch 16/80

27/27  3s 105ms/step - loss: 1.4225e-04 - mae: 0.0089 - val_loss: 2.3380e-04 - val_mae: 0.0124 - learning_rate: 1.2500e-04

Epoch 17/80

27/27  4s 141ms/step - loss: 1.3883e-04 - mae: 0.0091 - val_loss: 2.2337e-04 - val_mae: 0.0121 - learning_rate: 1.2500e-04

Epoch 18/80

27/27  4s 101ms/step - loss: 1.4270e-04 - mae: 0.0090 - val_loss: 2.7750e-04 - val_mae: 0.0136 - learning_rate: 1.2500e-04

Epoch 19/80

27/27  3s 102ms/step - loss: 1.4388e-04 - mae: 0.0092 - val_loss: 3.3281e-04 - val_mae: 0.0151 - learning_rate: 6.2500e-05

Epoch 20/80

27/27  3s 101ms/step - loss: 1.3532e-04 - mae: 0.0090 - val_loss: 2.9262e-04 - val_mae: 0.0140 - learning_rate: 6.2500e-05

Epoch 21/80

27/27  5s 103ms/step - loss: 1.3770e-04 - mae: 0.0089 - val_loss: 2.5218e-04 - val_mae: 0.0129 - learning_rate: 6.2500e-05

Prediction using the Model

```
In [ ]: pred_scaled = model.predict(test_ds)
        pred_scaled = pred_scaled.reshape(-1, num_features)
```

6/6  1s 81ms/step

Inverse Transform the values to original ones

```
In [ ]: pred = scaler.inverse_transform(pred_scaled)
        y_test_orig = scaler.inverse_transform(y_test)
```

Metrics for measuring the performance of the model

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error
        import numpy as np

        rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
        mae = mean_absolute_error(y_test_orig, pred)
```

```
mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

print("LSTM RMSE:", rmse)
print("LSTM MAE:", mae)
print("LSTM MAPE:", mape)
```

LSTM RMSE: 81.15594014669382

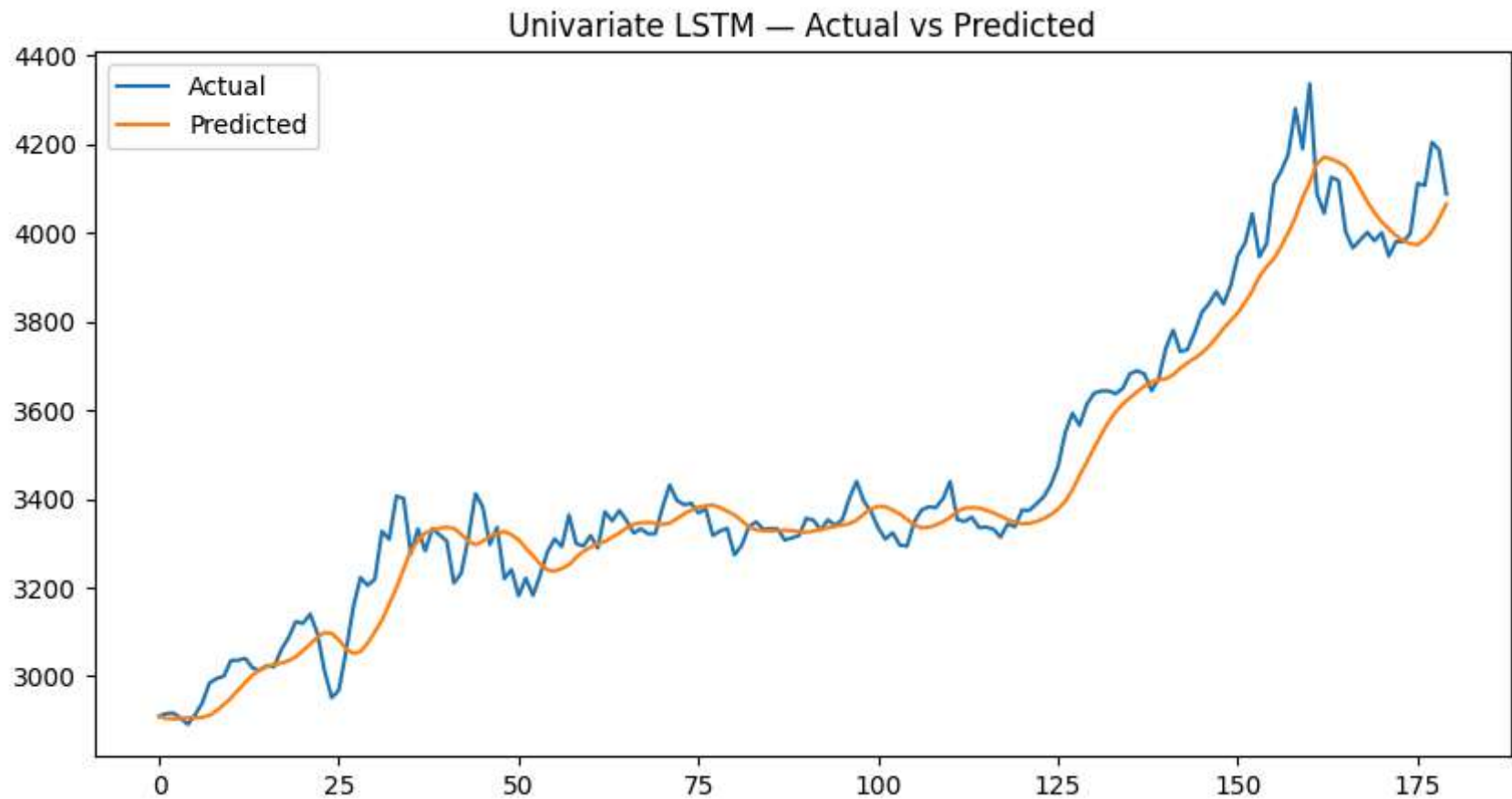
LSTM MAE: 61.81028747558594

LSTM MAPE: 1.7481202

Graph-Actual Vs Predicted

```
In [ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(y_test_orig, label="Actual")
plt.plot(pred, label="Predicted")
plt.title("Univariate LSTM - Actual vs Predicted")
plt.legend()
plt.show()
```



Comparison of the Metrics of all Three Models

```
In [ ]: import matplotlib.pyplot as plt

models = ["RNN", "GRU", "LSTM"]

rmse = [272.069384532696, 64.80756143793138, 81.15594014669382]
mae = [229.83074951171875, 49.33497619628906, 61.81028747558594]
mape = [6.356969, 1.4026474, 1.7481202]

colors = ["red", "green", "orange"]

plt.figure(figsize=(12,6))
```

```
# --- RMSE subplot ---
plt.subplot(1,3,1)
bars = plt.bar(models, rmse, color=colors)
plt.title("RMSE")
plt.ylabel("Error")
plt.grid(axis="y", linestyle="--", alpha=0.4)

# add value labels
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 2, f"{yval:.2f}",
             ha='center', fontsize=9)

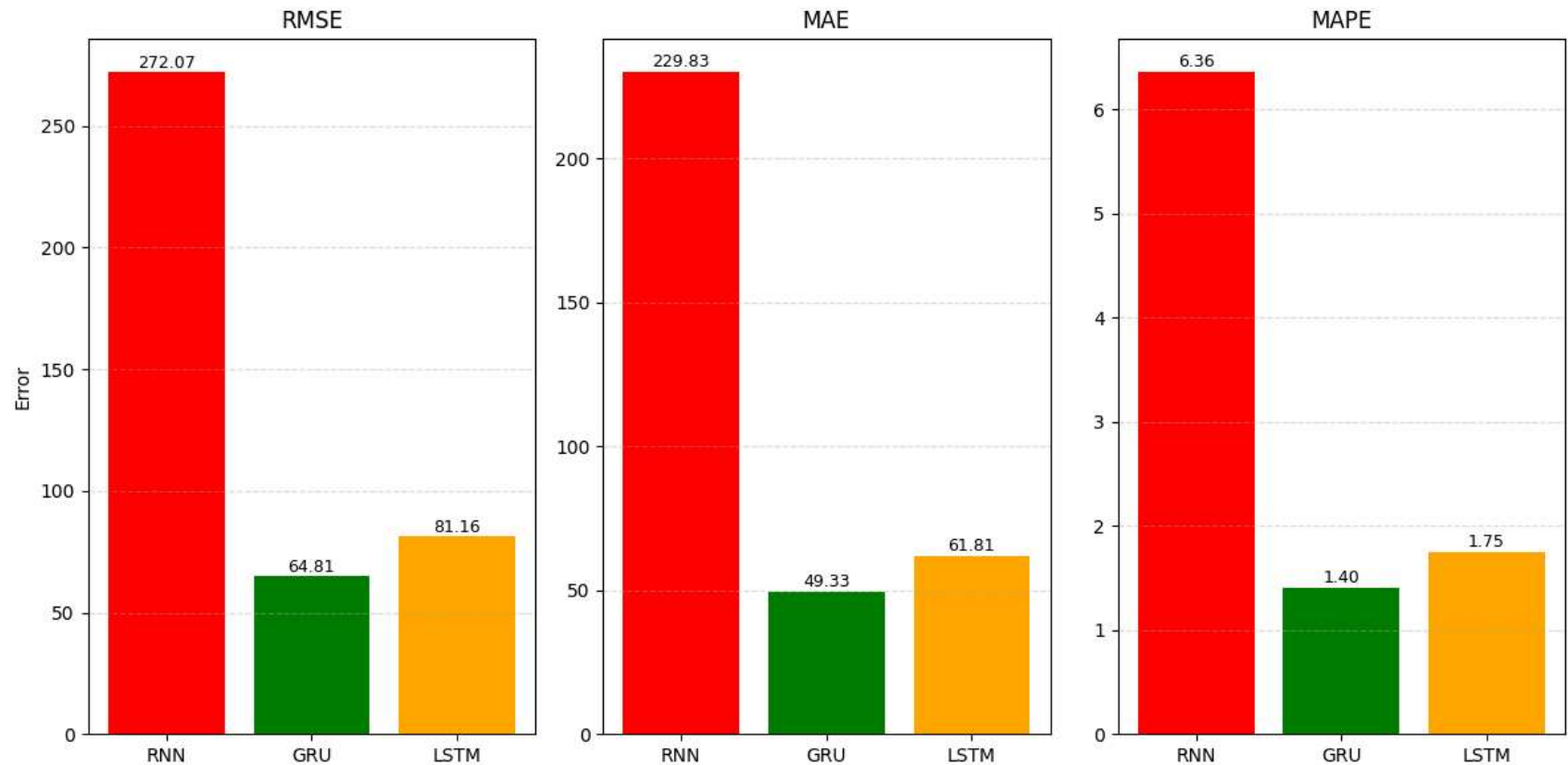
# --- MAE subplot ---
plt.subplot(1,3,2)
bars = plt.bar(models, mae, color=colors)
plt.title("MAE")
plt.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 2, f"{yval:.2f}",
             ha='center', fontsize=9)

# --- MAPE subplot ---
plt.subplot(1,3,3)
bars = plt.bar(models, mape, color=colors)
plt.title("MAPE")
plt.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.05, f"{yval:.2f}",
             ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```



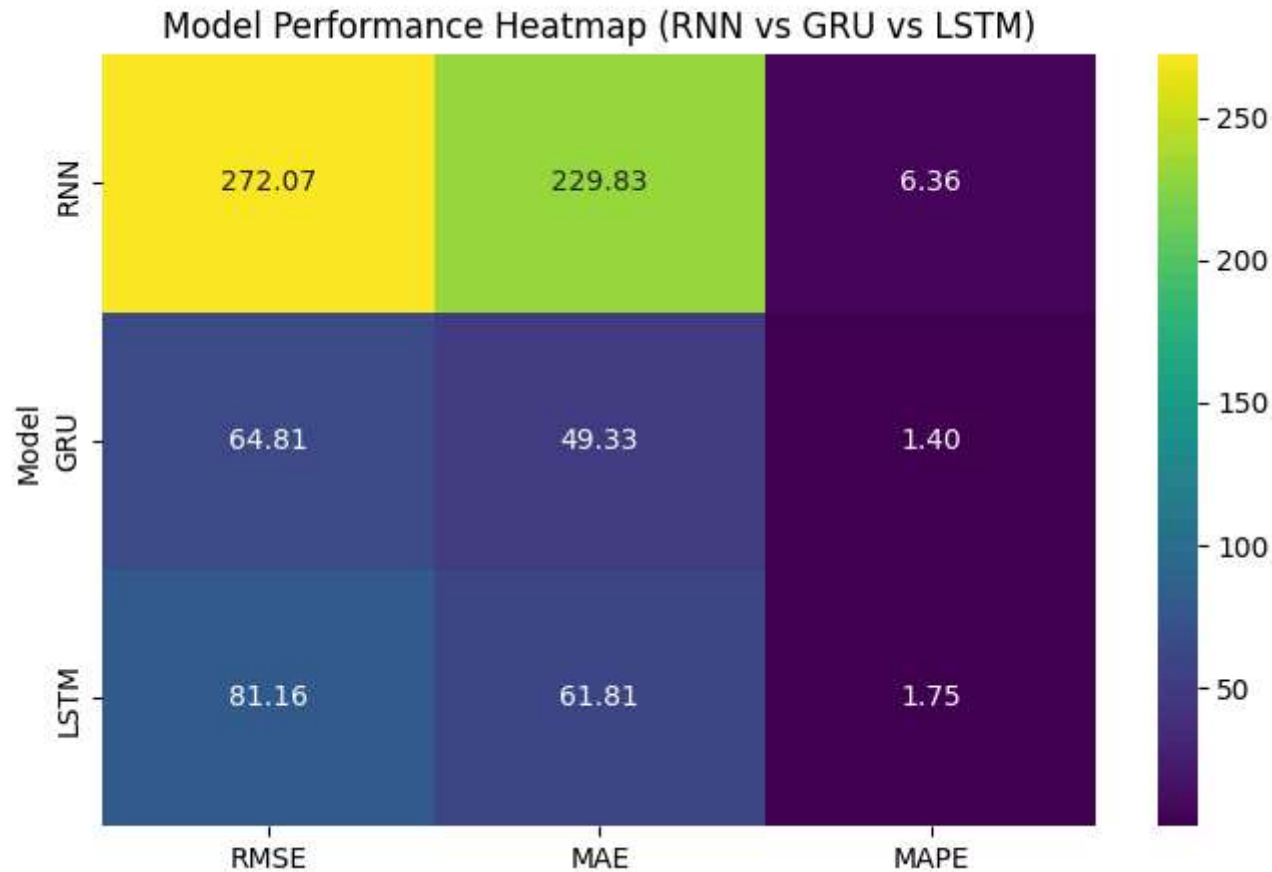
Confusion Matrix of the Metrics

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Your model metrics
data = {
    "Model": ["RNN", "GRU", "LSTM"],
    "RMSE": [272.069384532696, 64.80756143793138, 81.15594014669382],
    "MAE": [229.83074951171875, 49.33497619628906, 61.81028747558594],
    "MAPE": [6.356969, 1.4026474, 1.7481202]
}

df = pd.DataFrame(data).set_index("Model")
```

```
plt.figure(figsize=(8,5))
sns.heatmap(df, annot=True, cmap="viridis", fmt=".2f")
plt.title("Model Performance Heatmap (RNN vs GRU vs LSTM)")
plt.show()
```



- 1.Strong accuracy: With 1.75% MAPE, LSTM provides reliable forecasts suitable for planning and decision-making.
 - 2.Moderate absolute errors: RMSE 81 and MAE 62 indicate stable predictions with manageable deviations.
 - 3.Consistent performance: LSTM captures patterns well, making it useful for demand forecasting, budgeting, and resource planning.
- Overall: A solid, accurate model—though GRU still performs better for this dataset.

GRU is the best performer with the lowest errors (MAPE ~1.4%). It delivers the most accurate and stable forecasts, making it ideal for budgeting, demand forecasting, and operational planning.

LSTM performs well (MAPE ~1.7%) and provides reliable, consistent predictions, but is slightly less accurate than GRU.

RNN shows weaker accuracy (MAPE ~6.3% and high absolute errors). It is not suitable for business-critical forecasting.

Overall Recommendation: Deploy GRU for business decision-making—it offers high precision, minimizes forecast risk, and supports more confident strategic planning.